

Bengali Text Classification: A New multi-class Dataset and Performance Evaluation of Machine Learning and Deep Learning Models

Amartya Roy

Jadavpur University

Kamal Sarkar (✉ jukamal2001@yahoo.com)

Jadavpur University

Chintan Kumar Mandal

Jadavpur University

Research Article

Keywords: Text Classification, New Datasets, Machine Learning, Deep Learning, Bengali Language

Posted Date: July 4th, 2023

DOI: <https://doi.org/10.21203/rs.3.rs-3129157/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Bengali Text Classification: A New multi-class Dataset and Performance Evaluation of Machine Learning and Deep Learning Models

Amartya Roy¹, Kamal Sarkar^{1*}, Chintan Kumar Mandal¹

^{1*}Department of Computer Science and Engineering, Jadavpur University, Kolkata, 700009, West Bengal, India.

*Corresponding author(s). E-mail(s): jukamal2001@yahoo.com;
Contributing authors: amartyaroy1998@gmail.com;
chintankumar.mandal@jadavpuruniversity.in;

Abstract

This study focuses on Bengali text classification using machine learning and deep learning techniques. Text classification is a fundamental task in natural language processing (NLP) that involves categorizing text documents into predefined categories or classes. While text classification has received considerable attention in the English language, there is a limited amount of research specifically addressing Bengali text classification. This gap in the literature highlights the need for exploring and developing effective techniques tailored to the Bengali language. Furthermore, although some datasets for Bengali text classification tasks have been produced, most of the datasets have a limited number of labels. In our work, we introduce a new dataset for the Bengali text classification task, which has 38 class labels. The dataset includes data from the leading Bengali newspapers. We have evaluated many state-of-the-art machine learning and deep learning classification methods on our dataset to construct a benchmark that will facilitate future research.

Keywords: Text Classification, New Datasets, Machine Learning, Deep Learning, Bengali Language

1 Introduction

Text classification research is significantly impacted by the AI discipline of natural language processing (NLP). Over the past few decades, text classification has been extensively explored and handled in numerous applications [[1], [2], [3], [4], [5], [6], [7], [8]]

Text classification is an active research field. Many researchers are now interested in developing text mining applications that make use of text classification techniques. Most existing text classification algorithms have been tested on datasets in the English language. Studies on Bengali text classification are limited. Bengali, one of the most widely spoken languages in the world, has garnered significant attention in the field of natural language processing (NLP) and text classification. The increasing availability of digital content in Bengali has necessitated the development of effective techniques for organizing, analyzing, and extracting insights from Bengali textual data. Machine learning and deep learning approaches have emerged as powerful tools for Bengali text classification, enabling the automatic classification of Bengali documents into predefined classes.

Bengali text classification is a difficult task because Bengali is a morphologically rich language and the dimension of the feature vector becomes very high while the corpus is very large. Previous text research used datasets with few classes [9] and the features like the bag of words or n-grams [10]. Using these features classification algorithms like Naive Bayes (NB) [11], Support Vector Machine (SVM) [12], Hidden Markov Models (HMM) [13], Random Forests [14], etc. were used for text classification. However, the emphasis has switched to the use of deep learning techniques when large-scale datasets have been more accessible in recent years. Deep learning models no longer require extensive feature engineering and can be applied to a variety of tasks because they can learn representation from the data itself.

More recently, pre-trained language models based on transformers have been effectively applied to learn language representations using a significant amount of unlabeled input. BERT, RoBERTa, OpenAI GPT, and others are a few of these models. When optimized for various downstream tasks, such as text categorization [15], question answering [16], natural language inference [17], etc., these models have demonstrated to be incredibly successful. However, these models are typically trained on big multilingual datasets that may contain over a hundred languages or on large monolingual English corpora.

Motivated by these and the fact that, there have not been many works on Bengali text classification based on this approach, We have developed multiclass Bengali datasets for Bengali text classification. This dataset has 38 classes whereas the previously used public dataset has a relatively less number of classes. For example, T. Alam et al. [9] used a dataset that has 6 classes. We have also done a comprehensive study on Bengali text classification using both machine learning and deep learning techniques.

Our main contributions are summarized as follows: 1) We have introduced the new dataset for Bengali text classification. To our best knowledge, this is the Bengali dataset that has the largest number of classes.

2) For the purpose of building a benchmark to aid future research, on our dataset, we have evaluated various machine-learning and deep-learning methodologies using multilingual pre-trained Models. We have conducted a thorough analysis of the Bengali text classification problem based on comprehensive experimental results.

The organization of the paper is as follows: Section 2 represents Literature Survey on Bengali text classification. We have developed ML-based and DL-based text classification models for producing baseline results on Bengali datasets. The ML-based methods are discussed in Section 3 and the DL-based methods are discussed in Section 4. The detailed description of the dataset is presented in Section 5. We discuss experimental settings, evaluation metrics and results in Section 6. We conclude our work and discuss the future direction in Section 7.

2 Literature Survey on Bengali Text Classification

Text classification is emphasized in numerous articles, publications, and studies. In this section, we have discussed several prior research works that are pertinent to our work. Despite being one of the most widely spoken and culturally diverse languages with around 265 million native speakers, there hasn't been as much research on Bengali text classifications as there has been for English. The lack of labeled data for machine learning model training is the primary cause.

Because of its efficiency and simplicity in employing the joint probabilities of words and categories given a document, the Naive Bayes (Nave Bayes) probabilistic classifier is frequently used in text classification applications and experiments. Abu Nowshed Chy et al [18] applied Naive Bayes for the classification of Bangla news. Dhar et al. have published several research papers on text classification. In their earliest work, for Bangla text document classification, they have [19] used a Multinomial Naive Bayes classification approach with the TF-IDF-ICF feature selection method.

In the second paper of Dhar et al.[20], they used 5 classifiers, including MLP, RF, SVM, NBM, and K*. They have used term association and term aggregation feature extraction methods. For dimensionality reduction, they have used PCA. They evaluated their approach on a new dataset consisting of 8000 text documents taken from the online news corpus of Bengali publications, web pages, and websites. They considered eight different text categories and used WEKA for implementing the classifiers.

Islam1 et. al. [21] emphasized how feature selection can affect the Bengali text classification performance. They have used two feature selection techniques namely Chi-square distribution and normalized TF-IDF with traditional machine learning classifiers such as Support Vector Machine(SVM), Naïve Bayes(NB) and Stochastic Gradient Descent(SGD).

Mandal and Sen [22] employed four machine learning algorithms, including Support Vector Machine (SVM), Naive Bays, K-Nearest Neighbour (KNN), and Decision Tree for Bengali text classification. The dataset they used for their research was the BD corpus, which has 1000 documents and a total of 22,218 words. It is divided into five categories: business, sports, health, technology, and education. To extract important features from the documents, they used the TF-IDF feature extraction technique. They showed that SVM performed the best among all other classifiers they implemented.

There has been limited research on Bengali text classification using deep learning techniques. Our survey reveals that there are a couple of research papers in this direction. The authors of the paper [23] propose a text document classification system for the Bengali language using GloVe embedding and a Very Deep Convolution Neural Network (VDCNN) classifier. The authors developed two large datasets, a word embedding corpus (EC) and a text classification corpus (BDTC) for this classification task. The BDTC dataset has 13 categories which is much fewer than the categories we have considered for the development of our dataset. They also created an algorithm, called the Embedding Parameters Identification Algorithm, which selects the best embedding parameters for low-resource languages like Bengali. A total of 109 models are implemented using the combinations of embedding models and classification techniques (ML, CNN, sequential, BERT). In particular, 108 models are generated from three classification techniques where three ML-based techniques contributed to 36 models [12 (embedding model) $\times$ 3 (SVM, Libsvm & SGD)], three CNN-based techniques contributed to 36 models [12(embedding model) $\times$ 3 (CNN, DCNN, VDCNN)] and three sequential-based technique generated 36 models [12(embedding model) $\times$ 3 (LSTM, GRU, GRU + LSTM)]. One model is generated from the BERT-based technique (m-BERT). Among their proposed models, Glove+VDCNN achieved the highest accuracy.

The authors of the paper [24] applied five machine learning classifiers such as SVM, XGBoost, Logistic Regression, Multinomial NB, Random Forest, and two neural networks LSTM & MLP to categorize Bangla news articles, with the neural network LSTM performing the best. They have used TF-IDF for feature selection. Their dataset contains a total of 75951 samples and it is categorized into 12 classes.

In the research work [25], the authors have taken data from open resources which are containing the Bangla newspaper article corpus with 12 categories. For categorizing these article datasets, they have used word embedding methods to extract features from raw data and then used deep learning methods: Convolutional Neural Network (CNN), Artificial Neural Network (ANN), Long Short-term memory (LSTM) and Hybrid (CNN+Bi-LSTM) methods to classify them based on their contents. Next, they analyzed the performance of these deep learning-based classification models. Furthermore, the hybrid model has obtained a greater accuracy

Recently, large language models based on transformers [26] have achieved remarkable success in learning language representations through the utilization of vast amounts of data. Prominent examples of such models are OpenAI GPT [27], BERT [28], and RoBERTa [29]. These models have demonstrated exceptional performance when fine-tuned for various downstream tasks such as text classification, question answering, and natural language inference. Their effectiveness in these areas has been widely acknowledged. The authors of the paper [30] have experimented using six publicly available datasets with diverse topics including sentiment analysis(using 3 & 5 class Bengali dataset), emotion detection(using 5 class Bengali dataset), authorship attribution(using 14 class Bengali dataset) and news categorization(using 6 class Bengali dataset). They have used pre-trained models such as “BERT-base” “XLM-RoBERTa-base” & “XLM-RoBERTa-large” and showed the effectiveness of these approaches.

Compared to the existing work on Bengali text classification, our work differs in the development of a dataset with 38 categories. We have also evaluated most machine learning algorithms and transformer-based deep learning algorithms on our dataset. To prove the generalization ability of various machine learning and deep learning models, we have also tested them on another publicly available dataset.

3 Proposed Machine Learning Based Bengali Text Classification

In this section, we will present the machine learning-based models used for Bengali text classification. For this purpose, machine learning algorithms, Support Vector Machines (SVM), Random Forest, Logistic Regression, Decision Trees, KNN, and Naive Bayes have been used. These algorithms have been evaluated on our dataset consisting of Bengali text documents, and their performance has been evaluated using appropriate evaluation metrics.

The proposed ML-based classification approach has several steps such as 1) Preprocessing, 2) Document Representation 3) Classification

3.1 Preprocessing

Our pre-processing process is divided into two steps: Stopword Removal and Stemming.

Stopword Removal: Stop words are low-content words that occur frequently in a language. For example(in Bengali) অবশ্য, অনেক, অনেকে,অনেকেই,একে,একটি,এখন etc . The reason for stop word removal is to help a classification algorithm to focus on the important words when it classifies documents. We have used the NLTK library for removing stop words.

Stemming : Inflection leads to out-of-vocabulary problems. Since Bengali is a highly inflectional language, we have used a Bengali stemmer to convert a word to its stem. Though stemming does not produce the real roots of words, it is useful

in document classification because it alleviates the data sparseness problem and reduces the vocabulary size. Table 1 shows a sample example of word stemming

Table 1: Example of Stemming

Original Words	After Stemming
এটাই	এটা
সেটাই	সেটা
হয়তো	হয়
হেসেছিলেন	হাসা

3.2 Document Representation

The machine learning algorithms cannot understand the text input directly. The texts need to be converted to numerical vectors. For document classification, each document needs to be represented as a vector that is fed to a classifier.

In our work, we have used the TF-IDF-based vector space model(VSM) for document representation. In this method, a document is transformed into an n-dimensional vector where n is the vocabulary size. The vocabulary is created by a list of unique words in the entire dataset. The value of the component of a document vector is the TF-IDF weight of the corresponding vocabulary word if the word is present in the document. If the vocabulary word is not present in the document, the value of the component is set to 0. The TF-IDF weight of a word is calculated as follows.

$$TFIDF(w, d, D) = TF(w, d) * IDF(w, D)$$

where $TF(w, d) = \log(count_{w,d} + 1)$, $count_{w,d}$ = Number of times the word w occurs in document d, IDF(Inverse Document Frequency) is computed using the formula:

$$IDF(w, D) = \log\left(\frac{N}{|d \in D : w \in d|}\right)$$

where

- N : corpus size
- $|d \in D : w \in d|$: how many documents contain the word w occurs.
- If the term is absent from the document, the IDF becomes undefined. To avoid this, we generally convert it to $1 + |d \in D : w \in d|$.

3.3 Traditional ML-based Classifiers

In this study, we have used traditional machine learning algorithms for Bengali text classification. Details of these ML algorithms will be discussed later in this section. The ML algorithms that we have used in our study are Support Vector Machines, Decision Trees, Random Forest, KNN, Logistic Regression, and Naive Bayes. In the subsequent sub-sections, we will discuss the above-mentioned ML algorithms in brief.

3.3.1 Support Vector Machine

Support Vector Machine (SVM) is a supervised learning model used for regression analysis and classification tasks. It aims to find a boundary, called a hyperplane, in a 2-dimensional space that separates data points belonging to different classes in a dataset. The goal is to find a hyperplane that maximizes the distinction between the data points of different classes in an n-dimensional space. The data points closest to this hyperplane are referred to as Support Vectors.

For binary classification, the hyperplane equation is given by:

$$w \cdot x - b = 0$$

Here, w represents the normal vector to the hyperplane, b is a threshold term, and x is the input instance. If the equation evaluates to a positive value, the instance belongs to one class, while if it evaluates to a negative value, the instance belongs to the other class.

Normally, SVM supports binary classification, but it can be extended to handle multiclass classification using two approaches: One-vs-One and One-vs-All.

In our work, we utilized the scikit-learn API, which employs the One-vs-One approach. This approach breaks down the multiclass problem into multiple binary classification problems, with each pair of classes having its own binary classifier. Therefore, for classifying data points from a dataset with n classes, the classifier employs $\frac{n \cdot (n-1)}{2}$ SVMs. In our case, we have two datasets with 6 and 38 classes, resulting in the usage of 15 and 703 SVMs, respectively.

3.3.2 Random Forest

The random forest algorithm is a popular supervised learning method that generates a forest consisting of multiple trees. It can be used for regression and classification problems, although it is commonly applied to solve classification problems. This approach involves creating a collection of decision trees where the best feature is selected from a random subset of features for each tree. Random forest is composed of these trees, which collectively classify new objects based on a given input vector.

In random forest, all the decision trees are utilized to classify data. Each tree "votes" for the class and the final classification is determined by the majority vote from all the trees. However, there are two factors that can contribute to potential errors in random forest. Firstly, if there is a correlation among trees within the forest, the error rate may increase. Secondly, each tree carries its own weight, meaning a tree with a lower error rate is considered more powerful as a classifier.

3.3.3 Logistic Regression

Logistic regression is one of the well-renowned supervised methods for text categorization. LR was frequently utilized in the realm of data retrieval. In past research, Logistic regression has been investigated in the machine learning area, specifically in

English text categorization. In Logistic Regression the main focused word is "logistic" which refers to a logistic function that performs classification operations in the algorithm. In addition, the Logistic function is the foundation of the logistic regression model. The following figure (3.1) [36] shows a logistic or sigmoid curve and the equation 1 is:

$$f(x) = \frac{M}{1 + e^{-k(x-x_0)}} \quad (1)$$

Here, e = Euler's number, x_0 = x value of the sigmoid's midpoint, M = maximum value of curve, k = steepness or logistic rate of the curve and $f(x)$ = function output.

Logistic Regression is used to classify categorical dependent data by utilizing predictor variables. It converts probability scores from categorical dependent variables, hence creating a connection between a categorical variable which is dependent and a continuous variable which is independent. Binary logistic regression and multinomial logistic regression are two forms of logistic regression. The primary distinction between these two sorts is the type of labels used in them. Multinomial logistic regression is performed when the labels include several values. The model's of fundamental assumption is that the independent and dependent variables do not share a linear relationship [9]. When it comes to text classification the Logistic regression model identifies a vector including variables, assesses each input variable coefficients, and then assumes the text class using a word vector

3.3.4 KNN

K-Nearest Neighbors (KNN) is a non-parametric and instance-based classification algorithm. It operates based on the principle that similar instances tend to belong to the same class. In KNN, the "k" refers to the number of nearest neighbors used to make a prediction. Given a new input, KNN finds the k closest labeled instances in the training data based on a distance metric (e.g., Euclidean distance) and assigns the majority class label among those neighbors as the predicted label for the new instance. KNN is simple to implement, but its performance can be sensitive to the choice of k and the distance metric.

3.3.5 Decision Tree

A Decision Tree is a supervised machine learning algorithm used for classification and regression tasks. It creates a tree-like model where each internal node represents a feature or attribute, each branch represents a decision rule, and each leaf node represents the predicted class or value. Decision Trees partition the feature space based on the values of input features and recursively split the data into smaller subsets until a stopping criterion is met (e.g., purity of classes or depth of the tree). The splitting process is guided by various impurity measures such as Gini Index or Information Gain. Decision Trees are interpretable and can handle both categorical and numerical features. However, they may suffer from overfitting and can be sensitive to small changes in the training data

3.3.6 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem and assumes that the features are conditionally independent given the class label. It is known as "naive" because this assumption of independence is usually not true in real-world scenarios. Despite this simplification, Naive Bayes can perform well and is computationally efficient. It calculates the probability of each class for a given input by multiplying the prior probabilities of the classes with the conditional probabilities of each feature given the class. The class with the highest probability is assigned as the predicted class. Naive Bayes is particularly effective when there is a large number of features and relatively small training data. It is commonly used for text classification tasks, such as spam detection or sentiment analysis.

4 Proposed Deep Learning Based Bengali Text Classification Models

Deep learning has gained significant attention in natural language processing tasks, including text classification. In this section, we will focus on deep learning-based models for Bengali text classification. Deep learning models have the ability to automatically learn intricate patterns and representations from text data, making them suitable for complex classification tasks. We will explore word embedding-based and BERT-based deep learning models for document classification.

The proposed DL-based approach has several steps: 1) Preprocessing 2) Document Representation using word embedding or pre-trained language models 3) Document Classification

For deep learning-based text classification, the preprocessing step is simple. We have only used stopwords removal at this step. We discuss the other steps of the proposed DL-based approach in the subsequent subsections.

4.1 Document Representation using word embedding or pre-trained language models

We have used two types of document representation using deep learning models: (1) word embedding-based and (2) BERT-based. After representing documents as vectors using above mentioned two methods, they are labeled with the labels of the corresponding documents. Finally, the support vector classifier and BERT classifier are used for document classification. We have experimented with the various types of BERT classifiers such as Multilingual BERT, XLM-RoBERTa, etc. for finding out the suitable model for Bengali text classification. In this subsection, we discuss two different deep-learning methods for document representation.

4.1.1 Word embedding-based models

Word embedding refers to a representation of words in a way that captures their distributional properties. In this representation, each word is mapped to a shared

low-dimensional space and associated with a d -dimensional vector. Unlike many other word embeddings, FastText [31] takes into account the morphology of words. It achieves this by using continuous skip-grams, where each word is represented as a collection of character n -grams. For example, if we consider $n = 3$, the word "marvelous" would be represented as "<mar, arv, rve, vel, ell, llo, lou, ous>". This approach preserves subword information and enables the computation of valid word embeddings for out-of-vocabulary words. Hence, FastText can generate vectors for words that were not encountered during the training of word embeddings.

The learning process of word representations in FastText is based on the continuous skip-gram model introduced by Mikolov [32]. This model is simple and performs well even with limited training data. However, it overlooks the internal structure of words. To address this limitation, the creators of FastText propose different scoring functions that take into account the subword information.

For document representation, vectors of the words contained in the document (stop words are not considered) are derived from FastText. Finally, the document vector is obtained by averaging those word vectors as follows:

$$\text{Document Vector} = \frac{1}{N} \sum_{i=1}^N \text{Word Vector}_i$$

where: - Document Vector represents the vector representation of the document. - N is the total number of words in the document. - Word Vector $_i$ represents the vector representation of the i -th word in the document.

4.1.2 Pretrained Language Models

Pretrained language models have revolutionized the field of NLP by providing powerful representations of text, reducing the need for extensive hand-crafted features, and achieving state-of-the-art results on various benchmarks. They have been widely adopted and used in academia and industry for a wide range of NLP applications, including chatbots, language translation, content generation, sentiment analysis, and more. These models have significantly advanced the capabilities of natural language understanding and generation systems. Pretrained language models, such as Transformer, BERT [28] (Bidirectional Encoder Representations from Transformers), GPT [27] (Generative Pretrained Transformer), and RoBERTa [29] (Robustly Optimized BERT Approach), have achieved remarkable success in a wide range of natural language processing (NLP) tasks. These models capture rich contextual information, enabling them to understand the meaning of words in the context of a sentence or document. In our experiments, we utilize transformer language models [Vaswani et al. [26] & Dong et al. [33]] that are publicly accessible through HuggingFace's transformer library.

To adapt the transformer model discussed above, for the classification task, we incorporate a special start of sequence (SOS) token¹ at the beginning and a special end-of-sequence (EOS) token at the end.

We have used two variants of Multilingual BERT: Multi-Lingual BERT Multi-Lingual DistilBERT. Both models are trained on the Wikipedia corpus in 104 different languages. Multi-Lingual DistilBERT has 6 layers, 12 heads, and 134M parameters whereas Multi-Lingual BERT has 6 layers, 12 heads, and 177M parameters. On average, Multi-Lingual DistilBERT is twice as fast as Multi-Lingual BERT.

XLm-RoBERTa is a multilingual version of RoBERTa. It is pre-trained on 2.5TB of filtered CommonCrawl data containing 100 languages. For XLm-RoBERTa, we have also used two variants: XLm-RoBERTa-Base and XLm-RoBERTa-Large. XLm-RoBERTa-Base has 36 layers and 250M parameters whereas XLm-RoBERTa-Large has 48 layers and 560M parameters.

A brief description of general Multilingual BERT and XLm-RoBERTa is as follows.

i) **Multilingual BERT**- BERT is a language model that learns distributed representations from unlabeled texts by considering both the left and right contexts of a token. It utilizes the encoder component of the Transformer architecture. During the language model pretraining phase, two objective functions are employed. The first one is the masked language model (MLM), which randomly masks a certain percentage (15%) of input tokens, aiming to predict the original token’s vocabulary ID in that position. The bidirectional nature of BERT allows it to effectively utilize both preceding and succeeding tokens for this task. The second objective is the next sentence prediction (NSP) task, which involves binary classification to predict if two sentences are consecutive in the original text. Positive sentence pairs are created by selecting consecutive sentences, while negative sentence pairs are formed by combining sentences from different documents.

The multilingual variant of BERT (mBERT) is trained on a diverse set of over 100 languages using the largest Wikipedia corpus. Since the number of Wikipedia entries varies across languages, data sampling is performed with an exponentially smoothed weighting (with a factor of 0.7). This ensures that high-resource languages like English are under-sampled compared to low-resource languages. Additionally, word counts are sampled in a similar manner to ensure that low-resource languages have an adequate number of words in the vocabulary.

ii) **XLm-RoBERTa**- RoBERTa, an improved version of BERT, enhances its performance by training on larger datasets, utilizing a larger vocabulary, and incorporating longer sequences and larger batches during training. In this improvement, the next sentence prediction (NSP) task is removed, and only the masked language model (MLM) loss is utilized for pretraining.

XLm-RoBERTa, on the other hand, is the multilingual variant of RoBERTa, which is trained using a multilingual MLM. It is trained on a vast amount of data from

¹For BERT the SOS and EOS tokens are respectively [CLS] and [SEP]. For XLm-RoBERTa these are <s> and </s>

over one hundred languages, specifically more than two terabytes of filtered Common Crawl data. XLM-RoBERTa has demonstrated impressive performance across various multilingual natural language processing tasks and can achieve comparable results to monolingual language models.

We have used BERT tokenizer and encoder for document representation. Since our task is Bengali document classification and our dataset contains relatively longer documents, we face the problem of input length restriction imposed by both BERT models discussed above. To address the input length constraint of BERT (it can process a maximum of 512 input tokens at a time), we have developed a sliding window-based document segmentation algorithm which is discussed in **Algorithm 1**, that allows us to process documents with token lengths exceeding the limit of 512 tokens. In our dataset, we observed that a significant number of documents contain more than 512 tokens, making it necessary to employ this approach. The segmentation algorithm involves dividing the long documents into smaller segments using a sliding window technique. We set the window size to 512 tokens, which is the maximum length accepted by BERT, and use a stride of 256 tokens. By applying this sliding window approach, we ensure that we cover the entire length of the document while staying within the allowable token limit. For documents that can fit within the 512-token limit, we directly provide them as input to BERT. However, for those documents that exceed this limit, we divide them into overlapping segments using the sliding window. Each segment is then separately processed by BERT, and the resulting word embeddings are collected. To obtain the final representation of word embeddings for the long documents, we take the average of the embeddings of the segments. Thus a document is finally converted into a vector.

Algorithm 1: BERT-based document representation

Input : Text document, d
Output: Document Vector, doc_vec

```
1  $Tokens \leftarrow \text{BERT\_Tokenizer}(d)$  ▷ Tokenize the documents
2 if  $|Tokens| \leq 512$  then
3    $doc\_vec \leftarrow \text{BERT\_Encoder}(Tokens)$  ▷ Encode the tokens to a vector
4 else
5    $Segments \leftarrow \text{Break\_segments}(d)$  ▷ Break d into overlapping segments
6    $emb \leftarrow \text{zero\_vector}$  ▷ Initialize a zero vector of length 768
7    $i \leftarrow 0$  ▷ Initialize a counter
8   for  $s$  in  $Segments$  do
9      $Tokens \leftarrow \text{BERT\_Tokenizer}(s)$ 
10     $Seg\_vec \leftarrow \text{BERT\_Encoder}(Tokens)$ 
11     $emb \leftarrow emb + Seg\_vec$ 
12     $i \leftarrow i + 1$ 
13  end for
14   $doc\_vec \leftarrow \frac{1}{i} \cdot emb$  ▷ Calculate average vector
15 end if
16 return  $doc\_vec$ 
```

4.2 Classification

We have developed two kinds of deep learning-based text classification models, (1) word embedding-based text classification model and (2) pre-trained language model-based classification model. In the case of the word embedding-based text classification model, we have combined word embedding document representation with a support vector classifier(SVC) and, in the case of the pre-trained language model-based classifier, we have used BERT-based document representation (the procedure is discussed earlier in this paper) and softmax classifier. The brief descriptions of these two kinds of Bengali classification models are presented in the subsequent subsections.

4.2.1 Word Embedding + SVC

For this classification model, we have used FastText word embedding for document representation (which is already discussed in section 4.1.1) and a linear support vector(SVC) classifier to predict the class labels. The model architecture is shown in Figure 1.

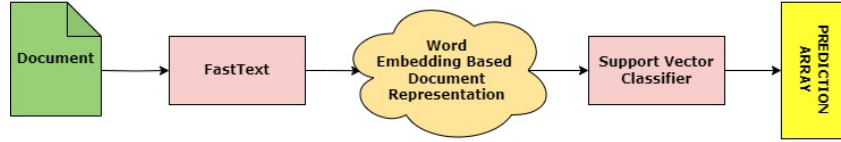


Figure 1: Word Embedding Model

4.2.2 BERT-Based Classifiers

As we have mentioned earlier, sliding window-based document segmentation and the above-mentioned BERT encoders have been used for document representation. In this process, we obtain an average vector of length 768 for each document. These vectors are labeled with the class label of the corresponding training document. Then the labeled vectors are used to train a softmax classifier. A general architecture of the BERT classifier is shown in Figure 2, although our implementation is different from this architecture because we have used the BERT encoder separately.

Since we have used four variants of BERT models, namely Multi-Lingual BERT, Multi-Lingual Distill BERT, XLM-RoBERTa-Base, and XLM-RoBERTa-Large, we have four BERT-based classifiers where each classifier differs in which variant of BERT is used.

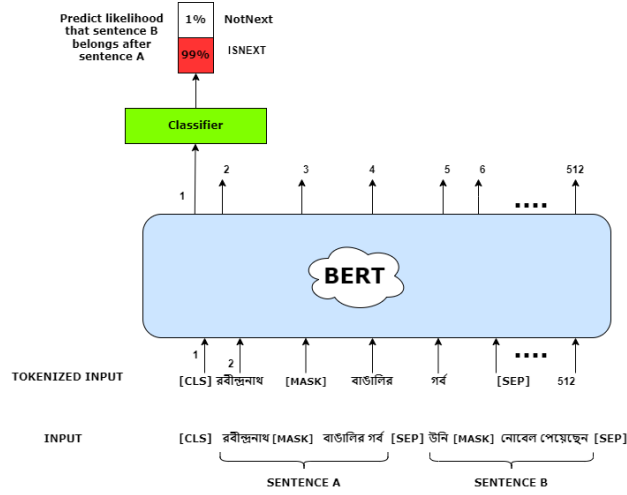


Figure 2: General Architecture Of BERT For Text Classification

5 Description of Datasets

We have done our experiments using two datasets. One dataset is developed by us. For the second dataset, we have used a public Bengali News Classification Dataset [9]. We have named these datasets respectively as Dataset-1 and Dataset-2 and have used this naming convention throughout this paper.

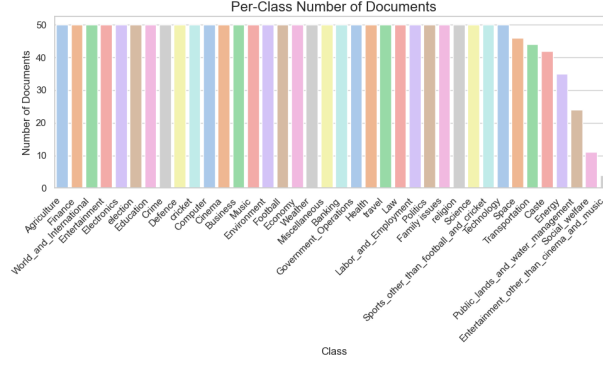
5.1 Dataset-1

This is the dataset developed by us. This dataset contains a total of 1756 documents and 38 classes namely 'Agriculture', 'Banking', 'Business', 'Caste', 'Cinema', 'Computer', 'cricket', 'Crime', 'Defence', 'Economy', 'Education', 'election', 'Electronics', 'Energy', 'Entertainment', 'Entertainment_other_than_cinema_and_music', 'Environment', 'Family issues', 'Finance', 'Football', 'Government_Operations', 'Health', 'Labor_and_Employment', 'Law', 'Miscellaneous', 'Music', 'Politics', 'Public_lands_and_water_management', 'religion', 'Science', 'Social_welfare', 'Space', 'Sports_other_than_football_and_cricket', 'Technology', 'Transportation', 'travel', 'Weather', 'World_and_International'. Each class-wise document distribution is shown in Figure 3a.

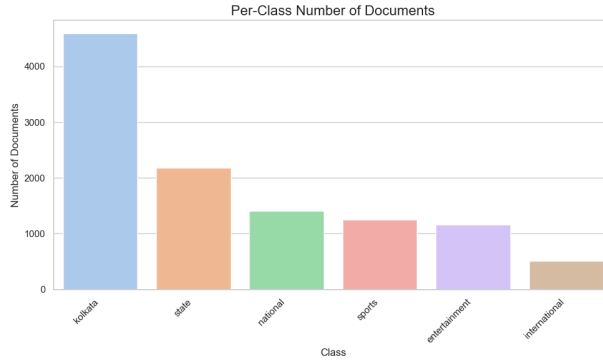
This dataset is randomly split into train and test sets in a ratio of 80:20.

5.2 Dataset-2

This is a publicly available dataset [34]. The dataset is available in two folders Train & Test. This dataset has 6 classes namely 'entertainment', 'state', 'sports', 'national', 'Kolkata', and 'international'. This dataset contains a total of 12,695 documents. The



(a) Documents per class for **Dataset-1**



(b) Documents per class for **Dataset-2**

Figure 3: Class-wise Distribution Over Each Dataset

training set contains 11284 documents whereas the test-set contains 1411 documents. The class-wise document distribution is shown in Figure 3b.

6 Experiments, Evaluations & Results

All experiments were conducted using Google Colab [35] and Kaggle² platforms. For dataset 1, we used the Google Colab platform and for dataset-2, we used the Kaggle platform.

We have developed 6 machine learning-based document classification models and 4 deep learning-based classification models. The performance of these models is tested on both datasets. The two widely used evaluation metrics, accuracy and F1 score are used for performance evaluation.

²<https://www.kaggle.com/>

6.1 Evaluation Metrics

We have used the accuracy score and F1 score for evaluating classification performance. Both of these have been calculated using the Scikit-learn package [36]

The $F1$ score can be interpreted as a harmonic mean of the precision and recall, where an $F1$ score reaches its best value at 1 and worst score at 0. The relative contribution of precision and recall to the $F1$ score are equal. The formula for the $F1$ score is:

$$F1 = 2 \cdot \frac{(\text{precision} \cdot \text{recall})}{(\text{precision} + \text{recall})}$$

The weighted $F1$ score extends this concept to multi-class classification problems. It calculates the $F1$ score for each class individually and then computes the average, taking into account the class imbalance. The weighted $F1$ score assigns a weight to each class based on the proportion of samples belonging to the class. The weight is determined by the number of instances belonging to each class relative to the total number of instances. This ensures that the $F1$ score is calculated with consideration for the class distribution, giving more importance to the performance on classes with a larger number of instances. By incorporating class weights, the weighted $F1$ score provides a more comprehensive evaluation of a model's performance in situations where some classes may be underrepresented or have higher importance than others. It takes into account the overall performance across all classes while giving more weight to the classes with a larger presence in the dataset. When interpreting the weighted $F1$ score, a higher value indicates better overall performance, considering both precision and recall.

6.2 Hyperparameter Settings

We have trained our developed machine learning and deep learning classifiers on the above-mentioned two datasets. We have separately tuned each classification model using the GridsearchCV module of sklearn. The hyperparameter settings for which we have achieved the best results (on two datasets) for each machine learning-based classification model are given in Table 2 and 3 respectively.

Table 2: Hyper-parameters for ML Models for Dataset-1

Classifiers	Parameters
SVC	C=100, kernel= 'rbf', gamma = 0.001
Random Forest Classifier	n-estimators = 500, min samples split = 2
Multinomial Naive Bayes	alpha = 0
Decision Tree Classifier	criterion ='log_loss', min_samples_split = 5
K-Nearest Neighbors	n_neighbors = 7, algorithm = 'kd_tree'
Logistic Regression	penalty = l2, C = 5.0, solver='newton-cg'

Table 3: Hyper-parameters for ML Models for Dataset-2

Classifiers	Parameters
SVC	C=1, kernel= 'linear', gamma = 0.01
Random Forest Classifier	n_estimators = 100, min_samples_split = 5
Multinomial Naive Bayes	alpha = 1
Decision Tree Classifier	criterion ='log_loss', min_samples_split = 7
K-Nearest Neighbors	n_neighbors = 9, algorithm = 'kd_tree'
Logistic Regression	penalty = l1, C = 4.0

We have trained³ the deep learning models using cross-entropy loss criterion and Adam optimization algorithm. All models were trained for 10 epochs and 50 epochs for Dataset-1 and Dataset-2 respectively with learning rate: $2e^{-5}$, seed: 1000, and Batch Size: 64. The model with the best validation set performance is evaluated on the test dataset.

The performance of each developed model is measured in terms of accuracy and weighted F1 score. For model evaluation and comparison, 5-fold cross-validation performance has been considered and the results averaged over 5 folds are reported in this paper.

6.3 Results

We have tested the proposed ML & DL-based Bengali document classification models on two datasets namely Dataset-1 (our own dataset) & Dataset-2 (a public dataset) which are described earlier in Section 3 and Section 4 respectively. The obtained results on these datasets are discussed in this section.

6.4 Results On Dataset-1

The best results obtained for Dataset-1 are shown in Table 4. For Dataset-1, the model “FastText with SVC” achieved the highest accuracy of 0.9261 and the highest weighted F1-score of 0.92. In this case, the interesting finding is that the SVC model with FastText Embedding outperformed other models in terms of both accuracy and weighted F1-score.

6.5 Results On Dataset-2

The best results obtained for Dataset-2 are shown in Table 5. We can see from this table that the deep learning model, XLM-RoBERTa-Large + softmax classifier, attained the highest accuracy of 0.9512 and the highest weighted F1-score of 0.95. Although SVC+ FastText performed best on Dataset-1, we observe that this model performs poorly on Dataset-2. For Dataset-2, the deep learning model with XLM-RoBERTa-Large encoder performed the best. The possible reason may be that

³The models were trained on a single 16 GB NVIDIA Tesla P100 GPU

Table 4: Model Performance on Dataset-1

Dataset	Model	Accuracy	F1-score (weighted)
Dataset-1	FastText with SVC	0.9261	0.92
	FastText with Logistic Regression	0.9033	0.90
	FastText with K-Nearest Neighbors	0.8886	0.89
	FastText with Decision Tree Accuracy	0.8902	0.89
	FastText with Random Forest	0.9220	0.92
	FastText with Multinomial Naive Bayes	0.8210	0.82
	TF-IDF with Linear SVC	0.6981	0.67
	Multi-Lingual BERT	0.7118	0.72
	Multi-Lingual DistilBERT	0.6997	0.69
	XLM-RoBERTa-Base	0.7482	0.74
	XLM-RoBERTa-Large	0.78	0.77

Dataset-2 is almost 7 times larger than Dataset-1. Dataset-2 has 12,695 documents whereas Dataset-1 has 1756 documents.

Table 5: Model Performance on Dataset-2

Dataset	Model	Accuracy	F1-score (weighted)
Dataset-2	FastText with SVC	0.4485	0.44
	FastText with Logistic Regression	0.4133	0.41
	FastText with K-Nearest Neighbors	0.4606	0.46
	FastText with Decision Tree Accuracy	0.4002	0.40
	FastText with Random Forest	0.4589	0.46
	FastText with Multinomial Naive Bayes	0.4001	0.40
	TF-IDF with Linear SVC	0.75	0.75
	Multi-Lingual BERT	0.9175	0.91
	Multi-Lingual DistilBERT	0.8975	0.89
	XLM-RoBERTa-Base	0.9310	0.93
	XLM-RoBERTa-Large	0.9512	0.95

These results reveal that no single model is best for both datasets. However, the word embedding-based document representation or BERT-based document representation always performs better for Bengali document classification. We also observe that, for both datasets, the traditional approach that uses TF-IDF-based document representation and Linear SVC performs worse than the deep learning-based models implemented by us.

6.6 Comparison With Existing Work

Since Dataset-2 is a public dataset, some researchers have already used this dataset and reported results in their published papers. We have also tested our developed models on Dataset-2. In this section, we have given a comparative study between our models and other existing works that used Dataset-2,

As we have developed Dataset-1, there exist no prior results on this dataset. We have used a state of the art ML and DL algorithms for producing the baseline text classification results on this dataset. For this dataset, we have achieved the highest accuracy of 92.61% using “Fastext with SVC”.

For Dataset-2, we have shown a comparative study in Table 6 which shows that our proposed model using a single pre-trained model(BERT & XLM-ROBERTa) outperforms the existing models proposed by T. Alam et al, 2020 [9]. Although our proposed model has some similarities with the model proposed by T. Alam et al, our model differs from the existing model in the way document is represented. Since the BERT model has an input length restriction(512 tokens), we have used an overlapping window-based document segmentation method (see 1) that breaks a long input document into a collection of contextual segments which are individually represented by BERT. Finally, the segment vectors collected from the BERT encoder are averaged to obtain the final document vector. But the work presented by T. Alam et al. in their paper did not mention how the longer documents (containing more than 512 tokens) are represented. Most likely, for the above-stated reasons, our proposed models perform better than the existing models.

Table 6: Result on Dataset-2(news classification) dataset

	Model	Accuracy	F1 Score
Others	BERT-base [9]	0.9128	0.9128
	XLM-RoBERTa-base [9]	0.9270	0.9284
	XLM-RoBERTa-large [9]	0.9341	0.9340
Ours	BERT-base	0.9175	0.9175
	XLM-RoBERTa-base	0.9310	0.9310
	XLM-RoBERTa-large	0.9512	0.9512

6.7 Error Analysis

In this section, we will discuss the error analysis for each dataset. We will give an overview about which class is predicted most correctly, and which is less correctly predicted.

For Dataset-1, we have 38 classes which we have already discussed in section 5.

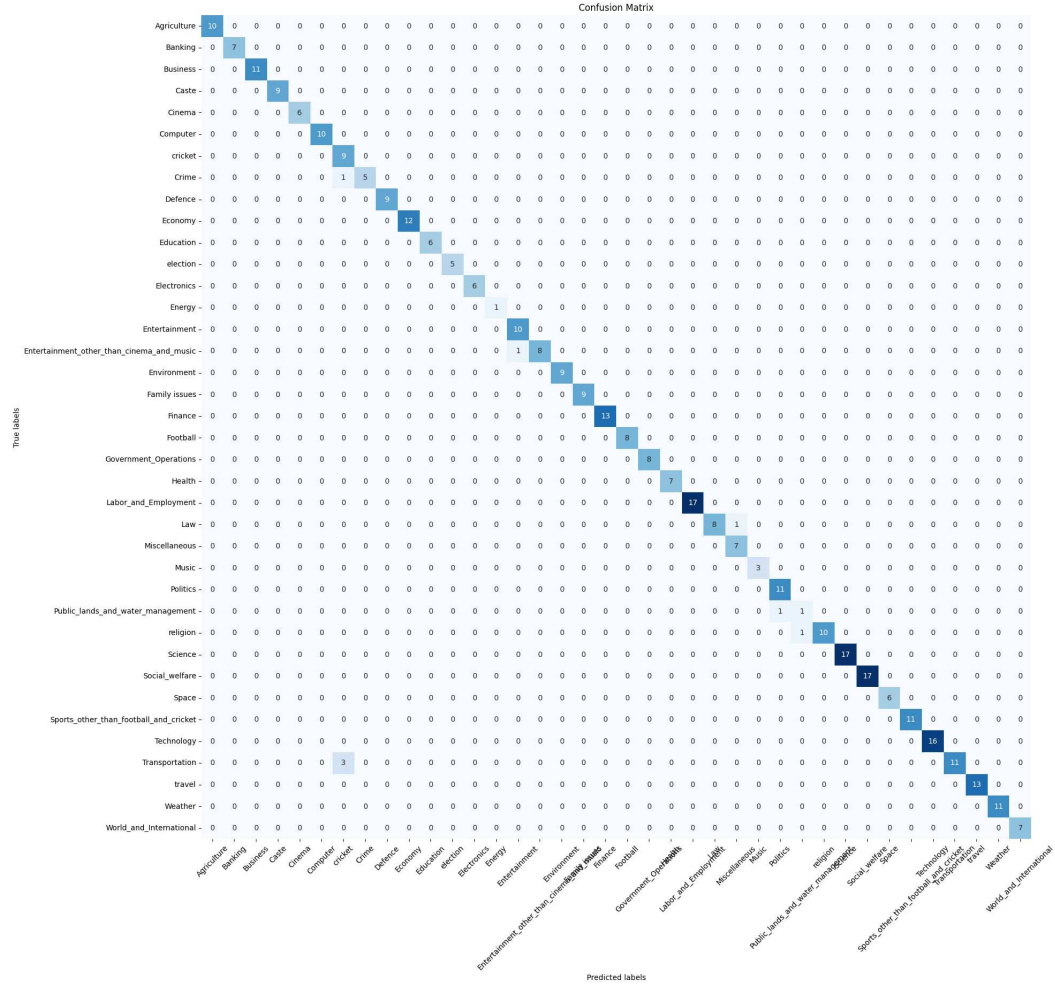


Figure 4: Confusion Matrix For Dataset-1

The confusion matrix of the best model for Dataset-1 is given in Figure 4. From this confusion matrix, we can conclude that most of the classes are predicted very accurately except a few classes namely {"Cricket" and "Transportation", }; {"Entertainment_other_than_cinema_and_music", and "Entertainment"}. After

inspection of Dataset-1, We observe that the model is highly confused to discriminate between these two classes namely 1) Entertainment_other_than_cinema_and_music and 2) Entertainment

For Dataset-2, there are 6 classes which we have discussed in section 5.

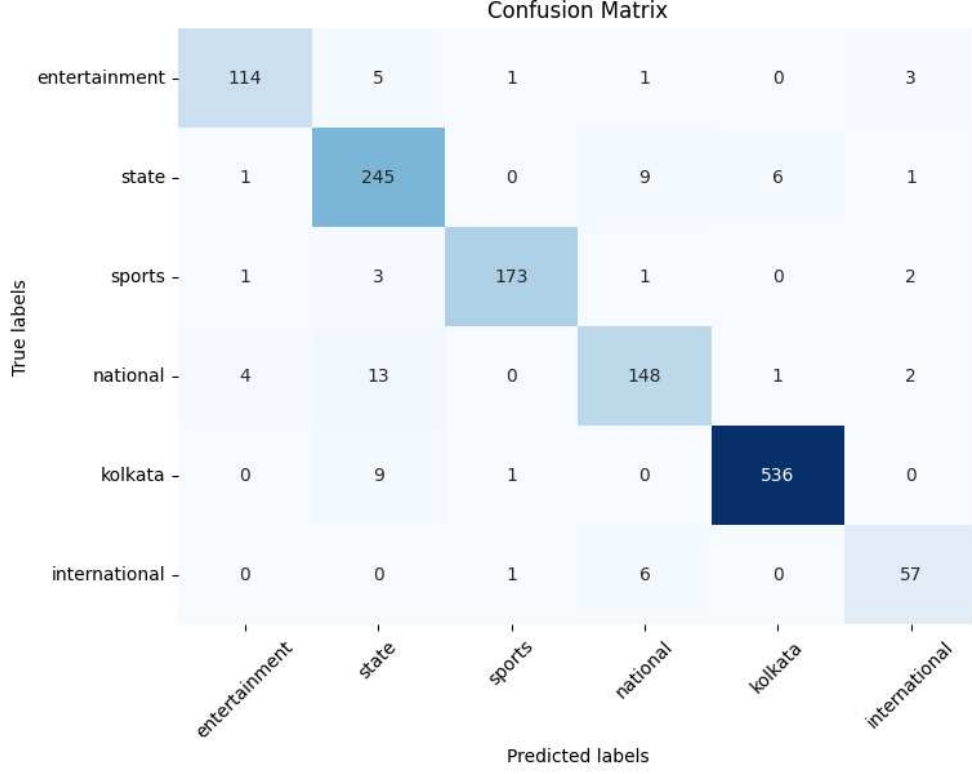


Figure 5: Confusion Matrix For Dataset-2

The confusion matrix of the best model for Dataset-2 is given in Figure 5. From the confusion matrix, we can conclude that the model performs poorly for the classes namely {"national" and "state"} and; {"kolkata" and "state"}. The possible reason is that these classes are highly overlapped.

7 Conclusion and Future Work

This study has undertaken a comprehensive exploration of Bengali text classification using machine learning and deep learning approaches. The research has contributed to the field by developing a new dataset and evaluating the efficacy of different machine

learning and multilingual pre-trained models for Bengali text classification tasks and constructing a benchmark on Bengali text classification.

Through extensive evaluation and analysis, the study has identified the most effective deep learning models for Bengali text classification, providing valuable insights for researchers and practitioners in developing high-performing classifiers for the Bengali language.

While this study has made significant contributions to Bengali text classification, there are several avenues for future research and improvement. Some potential areas for future work include an evaluation using larger datasets and exploring other deep learning architectures. In the future, our plan will be to increase the size of the dataset developed by us. For the deep architecture, our future plan is to combine multiple pre-trained models for Bengali document classification.

Although we have tested our models on multi-class document classification datasets, we hope that the outcomes of our work will have practical implications for various applications including sentiment analysis, document categorization, and information retrieval. With minor modifications, The proposed models can be applied to text classification task in other Indian languages like Hindi, Marathi, Tamil, etc.

Supplementary information. Not Applicable

Acknowledgments. No funding was received for this work.

Declarations

- Funding: No funding was received for this work
- Conflict of interest: The authors have no conflicts of interest.
- Ethics approval: Not applicable
- Consent to participate: Not applicable
- Consent for publication: Not applicable
- Availability of data and materials: We have conducted our experiments using our own dataset and a public dataset whose link is given in the paper. The link to our own dataset will be given in the camera-ready version of the paper.
- Authors' contributions : Conceptualization: Kamal Sarkar
Methodology: Amartya Roy, Kamal Sarkar
Software: Amartya Roy
Validation: Amartya Roy
Formal analysis: Kamal Sarkar, Amartya Roy, Chintan Kumar Mandal
Investigation: Amartya Roy, Kamal Sarkar
Resources: Kamal Sarkar
Data Curation: Kamal Sarkar, Amartya Roy
Writing - Original Draft : Amartya Roy, Kamal Sarkar
Writing - Review and Editing: Kamal Sarkar, Chintan Kumar Mandal
Visualization: Amartya Roy
Supervision: Kamal Sarkar, Chintan Kumar Mandal
Project administration: Kamal Sarkar, Chintan Kumar Mandal

Compliance with Ethical Standards

- Disclosure of potential conflicts of interest: The authors have no conflicts of interest to declare that are relevant to the content of this article.
- Research involving Human Participants and/or Animals: During the research, no human participants or animals were involved.
- Informed Consent: This article does not involve any studies with human participants or animals performed by any of the authors.

References

- [1] Jiang, M., Liang, Y., Feng, X., Fan, X., Pei, Z., Xue, Y., Guan, R.: Text classification based on deep belief network and softmax regression. *Neural Computing and Applications* **29**, 61–70 (2018)
- [2] Kowsari, K., Brown, D.E., Heidarysafa, M., Meimandi, K.J., Gerber, M.S., Barnes, L.E.: Hdltext: Hierarchical deep learning for text classification. In: 2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA), pp. 364–371 (2017). IEEE
- [3] McCallum, A., Nigam, K., *et al.*: A comparison of event models for naive bayes text classification. In: AAAI-98 Workshop on Learning for Text Categorization, vol. 752, pp. 41–48 (1998). Madison, WI
- [4] Kowsari, K., Heidarysafa, M., Brown, D.E., Meimandi, K.J., Barnes, L.E.: Rmdl: Random multimodel deep learning for classification. In: Proceedings of the 2nd International Conference on Information System and Data Mining, pp. 19–28 (2018)
- [5] Heidarysafa, M., Kowsari, K., Brown, D.E., Meimandi, K.J., Barnes, L.E.: An improvement of data classification using random multimodel deep learning (rmdl). arXiv preprint arXiv:1808.08121 (2018)
- [6] Lai, S., Xu, L., Liu, K., Zhao, J.: Recurrent convolutional neural networks for text classification. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 29 (2015)
- [7] Aggarwal, C.C., Zhai, C.: A survey of text classification algorithms. *Mining text data*, 163–222 (2012)
- [8] Aggarwal, C.C., Aggarwal, C.C.: *Mining Text Data*. Springer, ??? (2015)
- [9] Alam, T., Khan, A., Alam, F.: Bangla text classification using transformers. arXiv preprint arXiv:2011.04446 (2020)
- [10] Wei, Z., Miao, D., Chauchat, J.-H., Zhao, R., Li, W.: N-grams based feature selection and text representation for chinese text classification. *International Journal of Computational Intelligence Systems* **2**(4), 365–374 (2009)

- [11] Xu, S.: Bayesian naïve bayes classifiers to text classification. *Journal of Information Science* **44**(1), 48–59 (2018)
- [12] Joachims, T.: Text categorization with support vector machines: Learning with many relevant features. In: *European Conference on Machine Learning*, pp. 137–142 (1998). Springer
- [13] Yi, K., Beheshti, J.: A hidden markov model-based text classification of medical documents. *Journal of Information Science* **35**(1), 67–81 (2009)
- [14] Chen, H., Wu, L., Chen, J., Lu, W., Ding, J.: A comparative study of automated legal text classification using random forests and deep learning. *Information Processing & Management* **59**(2), 102798 (2022)
- [15] Sun, C., Qiu, X., Xu, Y., Huang, X.: How to fine-tune bert for text classification? In: *Chinese Computational Linguistics: 18th China National Conference, CCL 2019, Kunming, China, October 18–20, 2019, Proceedings 18*, pp. 194–206 (2019). Springer
- [16] Garg, S., Vu, T., Moschitti, A.: Tanda: Transfer and adapt pre-trained transformer models for answer sentence selection. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 7780–7788 (2020)
- [17] Zhang, Z., Wu, Y., Zhao, H., Li, Z., Zhang, S., Zhou, X., Zhou, X.: Semantics-aware bert for language understanding. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 9628–9635 (2020)
- [18] Chy, A.N., Seddiqui, M.H., Das, S.: Bangla news classification using naive bayes classifier. In: *16th Int’l Conf. Computer and Information Technology*, pp. 366–371 (2014). IEEE
- [19] Dhar, A., Dash, N.S., Roy, K.: Classification of bangla text documents based on inverse class frequency. In: *2018 3rd International Conference On Internet of Things: Smart Innovation and Usages (IoT-SIU)*, pp. 1–6 (2018). IEEE
- [20] Dhar, A., Mukherjee, H., Dash, N.S., Roy, K.: Performance of classifiers in bangla text categorization. In: *2018 International Conference on Innovations in Science, Engineering and Technology (ICISSET)*, pp. 168–173 (2018). IEEE
- [21] Islam, M.S., Jubayer, F.E.M., Ahmed, S.I.: A comparative study on different types of approaches to bengali document categorization. *arXiv preprint arXiv:1701.08694* (2017)
- [22] Mandal, A.K., Sen, R.: Supervised learning methods for bangla web document categorization. *arXiv preprint arXiv:1410.2045* (2014)
- [23] Hossain, M.R., Hoque, M.M., Siddique, N., Sarker, I.H.: Bengali text document

categorization based on very deep convolution neural network. *Expert Systems with Applications* **184**, 115394 (2021)

- [24] Salehin, K., Alam, M.K., Nabi, M.A., Ahmed, F., Ashraf, F.B.: A comparative study of different text classification approaches for bangla news classification. In: 2021 24th International Conference on Computer and Information Technology (ICCIT), pp. 1–6 (2021). IEEE
- [25] Alam, S., Haque, M.A.U., Rahman, A.: Bengali text categorization based on deep hybrid cnn–lstm network with word embedding. In: 2022 International Conference on Innovations in Science, Engineering and Technology (ICISSET), pp. 577–582 (2022). IEEE
- [26] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
- [27] Zhu, Q., Luo, J.: Generative pre-trained transformer for design concept generation: an exploration. *Proceedings of the Design Society* **2**, 1825–1834 (2022)
- [28] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018)
- [29] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019)
- [30] Alam, T., Khan, A., Alam, F.: Bangla text classification using transformers. *arXiv preprint arXiv:2011.04446* (2020)
- [31] Joulin, A., Grave, E., Bojanowski, P., Douze, M., Jégou, H., Mikolov, T.: Fast-text. zip: Compressing text classification models. *arXiv preprint arXiv:1612.03651* (2016)
- [32] Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J.: Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* **26** (2013)
- [33] Dong, Y., Cordonnier, J.-B., Loukas, A.: Attention is not all you need: Pure attention loses rank doubly exponentially with depth. In: International Conference on Machine Learning, pp. 2793–2803 (2021). PMLR
- [34] Kunchukuttan, A., Kakwani, D., Golla, S., Bhattacharyya, A., Khapra, M.M., Kumar, P., et al.: Ai4bharat-indicnlp corpus: Monolingual corpora and word embeddings for indic languages. *arXiv preprint arXiv:2005.00085* (2020)

- [35] Bisong, E.: Google Colaboratory, pp. 59–64. Apress, Berkeley, CA (2019).
https://doi.org/10.1007/978-1-4842-4470-8_7 . https://doi.org/10.1007/978-1-4842-4470-8_7
- [36] Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., Layton, R., VanderPlas, J., Joly, A., Holt, B., Varoquaux, G.: API design for machine learning software: experiences from the scikit-learn project. In: ECML PKDD Workshop: Languages for Data Mining and Machine Learning, pp. 108–122 (2013)